

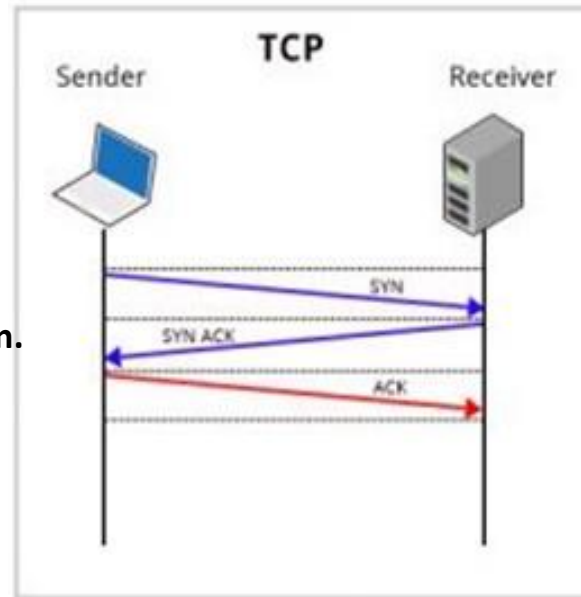
Universidad Tecnológica Nacional – Facultad Regional Rosario
Ingeniería en Sistemas de Información
Cátedra de Redes de Datos

CAPA DE TRANSPORTE
PROTOCOLO TCP (Transmission Control Protocol)

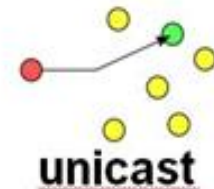
CAPA 4 - PROTOCOLO DE TRANSPORTE TCP

• ÍNDICE:

- 1- Introducción.
- 2 – Características.
- 3 – Encabezamiento.
- 4- Campos opcionales de encabezamiento.
- 5 – Establecimiento de la conexión.
- 6 – Secuencia de Estados de la comunicación.
- 7 – Ventana Deslizante.
- 8 – Temporizadores.
- 9 – Control de Congestión.
- 10 – “Acknowledgement” Selectivo.
- 11 - Bibliografía.



- **Slower but reliable transfers**
- **Typical applications:**
 - Email
 - Web browsing



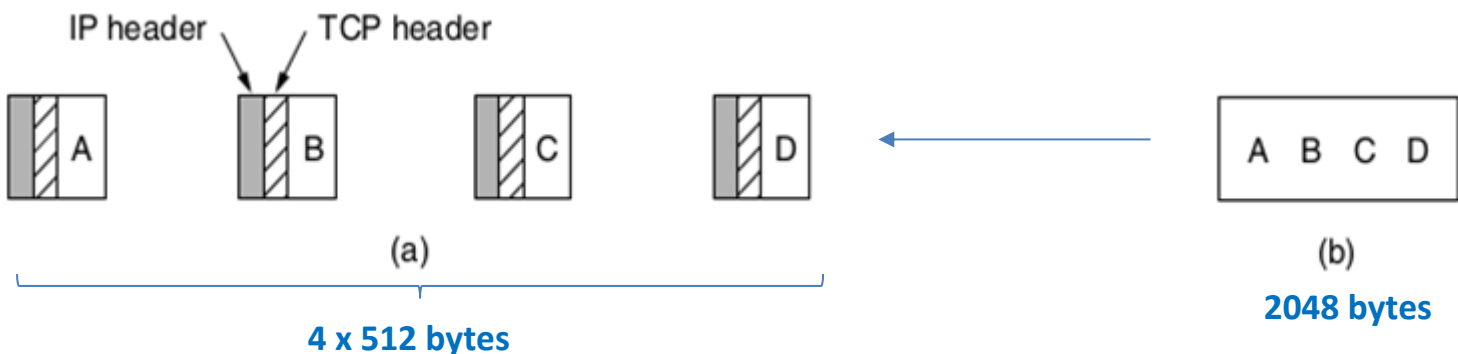
1 – INTRODUCCIÓN.

- **TCP (Transmission Control Protocol):** Fue diseñado para brindar una conexión confiable extremo a extremo atravesando redes poco confiables. Fue presentado en la RFC 793 de 1981, y diseñado para adaptarse a las distintas topologías de redes, anchos de banda, retardos, tamaño de paquetes y otras variaciones de parámetros que se pueden encontrar en internet. Posteriormente se introdujeron innumerables modificaciones y mejoras. Tal es así que la RFC 4614 es una guía de todas las RFC que intervienen en TCP.
- **Puertos:** Una entidad TCP se identifica por un puerto, igual que UDP, que es un campo de dirección de 16 bits. Los puertos con número menor a 1024 se reservan para aplicaciones comunes y usuarios con privilegios, (ejemplo: root en UNIX). Algunas de las aplicaciones más conocidas aparecen en la tabla de abajo. La lista completa puede obtenerse de www.iana.org, que es la institución que los registra. Actualmente hay más de 700 puertos asignados. Los puertos, de 1024 en adelante pueden usarse libremente. Por ejemplo, la aplicación BitTorrent utiliza desde el puerto 6881 al 6887, aunque también puede utilizar otros puertos. Estos procesos de transporte se ejecutan en segundo plano y se denominan “daemon”, (demonio). Por ejemplo, el “FTP daemon” usa los puertos 20 y 21.

Port	Protocol	Use
20, 21	FTP	File transfer
22	SSH	Remote login, replacement for Telnet
25	SMTP	Email
80	HTTP	World Wide Web
110	POP-3	Remote email access
143	IMAP	Remote email access
443	HTTPS	Secure Web (HTTP over SSL/TLS)
543	RTSP	Media player control
631	IPP	Printer sharing

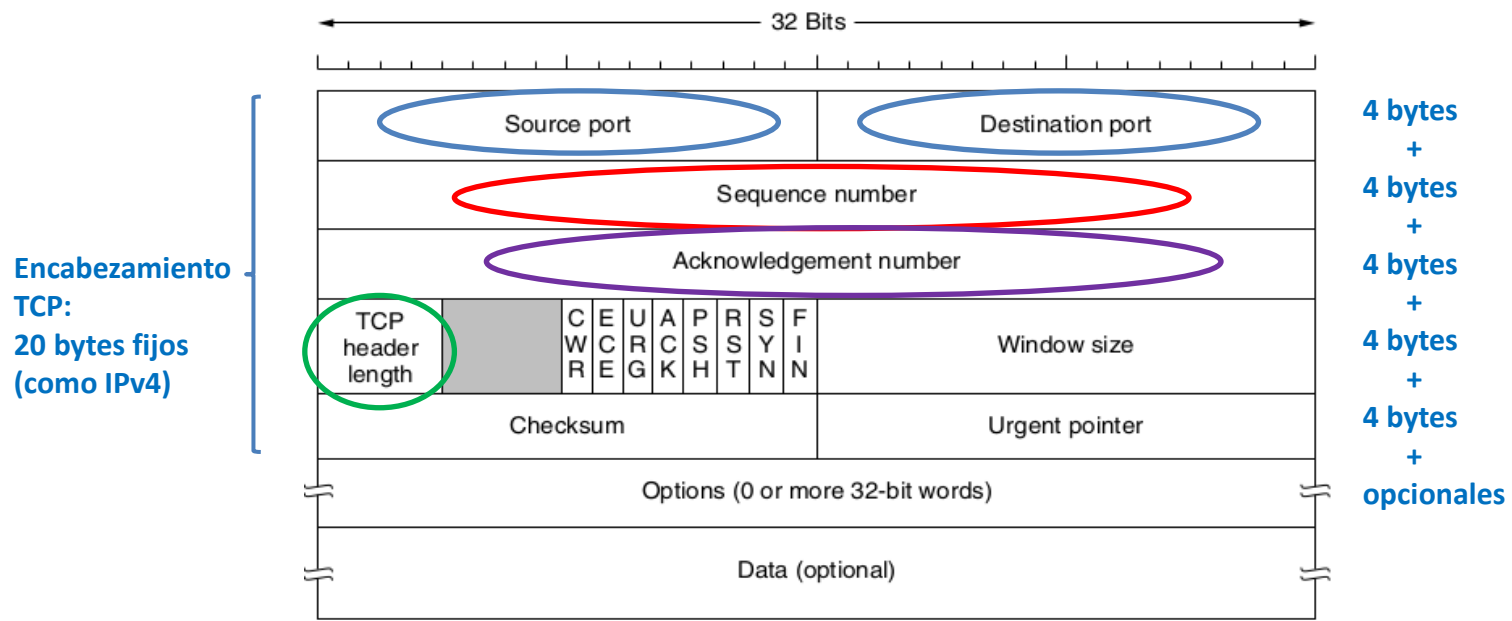
2 – CARACTERÍSTICAS TCP.

- **Full-dúplex unicast:** TCP comunica dos hosts entre sí, usualmente en arquitectura cliente-servidor. TCP no soporta multicasting o broadcasting. Todas las conexiones TCP son unicast, full-duplex, de extremo a extremo.
- **Tamaño:** Un segmento TCP puede extenderse hasta casi **64 KB**, aunque lo usual es que su tamaño no exceda 1,46 KB para poder ingresar en una trama Ethernet (1500 Bytes) con los encabezamientos de TCP/IP.
- **Urgencia:** Algunas aplicaciones necesitan actualizar datos periódicamente. Para ello, los segmentos TCP tienen un indicador de “urgente” para ser enviado lo antes posible, sin esperar el orden del buffer de salida. Y aunque las aplicaciones no pueden forzar a TCP a enviar datos “urgentes”, los sistemas operativos han desarrollado opciones para tal fin. Por ejemplo: el comando TCPNODELAY en Windows y Linux. De todos modos, este servicio “urgent data” de TCP es raramente utilizado por algunas diferencias con su implementación.
- **Byte-stream:** Una conexión TCP se dice que es “byte-stream” y no “mensaje-stream”. Esto quiere decir que, si un proceso escribe 2048 bytes de datos en 4 cadenas de 512 bytes, estos bytes pueden enviarse así tal cual, o en dos cadenas de 1024 bytes, o una sola de 2048 bytes, o alguna otra combinación, como se observa en la figura. Y el receptor tampoco tiene forma de saber cómo se han particionado los bytes al enviarlos por TCP.



3 – ENCABEZAMIENTO TCP.

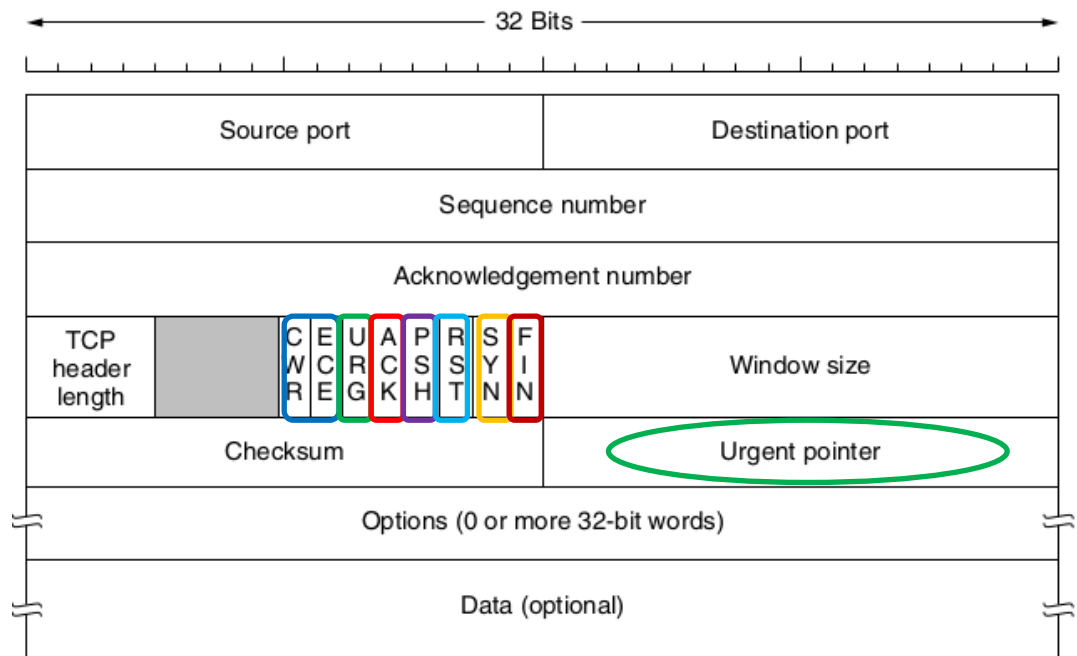
- La longitud total de un segmento TCP es casi 64 KB. Tiene la limitación de caber en el campo de carga de un paquete IP. Y la longitud máxima de un paquete IP es 64 KB incluido el encabezamiento. Además, en capa 2 tenemos el máximo MTU (maximum transmission unit) que en el caso de Ethernet son 1500 bytes. Si bien puede fragmentarse un segmento IP para entrar en la trama de capa 2, esto trae otros inconvenientes de performance que se busca evitar, por eso la longitud del segmento TCP no suele pasar los 1460 bytes, para no ser fragmentado.
- Los primeros campos:** Llevan los números de los puertos de origen y destino. Luego, el número de secuencia del segmento enviado y el del segmento recibido.
- TCP header Length:** El encabezamiento de un segmento TCP consta de 20 bytes fijos más otros bytes opcionales. En el campo de 4 bits que se observa en la figura se indica cuando comienzan los datos, ya que el encabezamiento puede contener algunas opciones que alargan su longitud. Los siguientes 4 bits (en gris) no se usan.



3 – ENCABEZAMIENTO TCP (continuación).

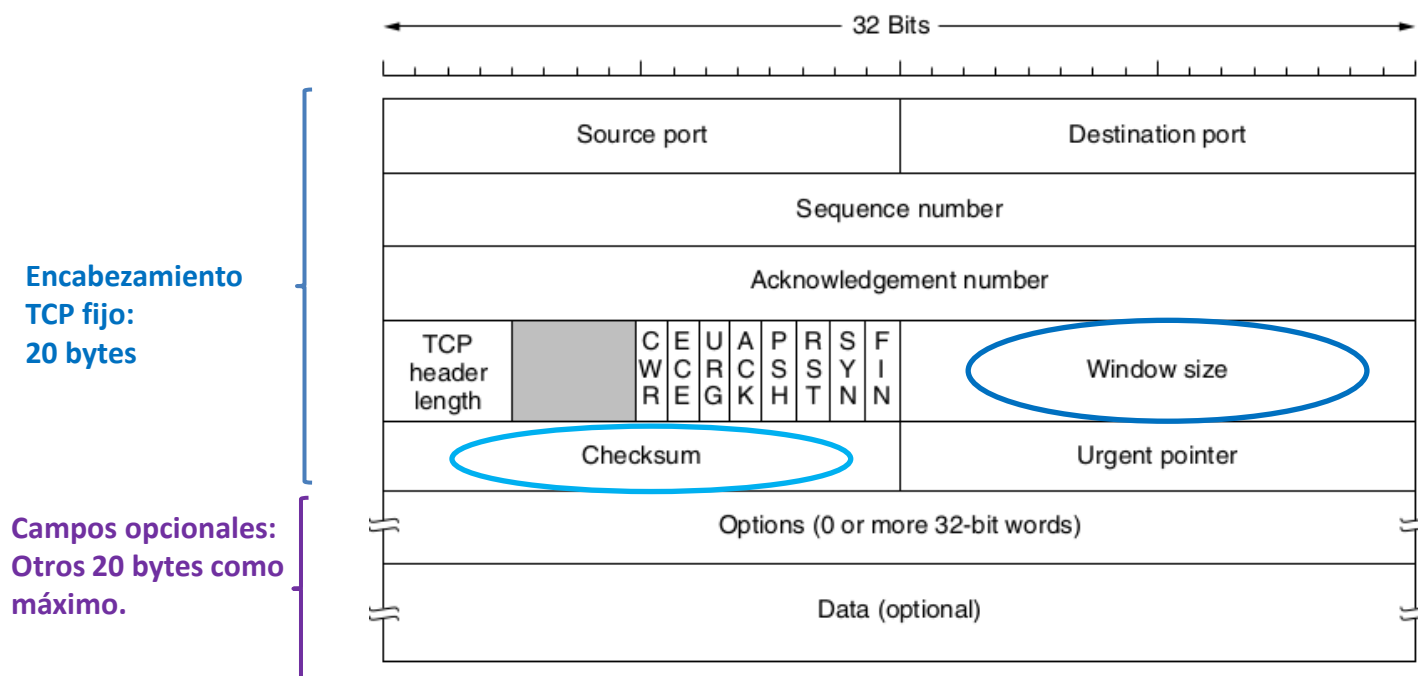
Los 8 bits indicados en rojo señalan varias cosas:

- **CWR y ECE:** Indican congestión cuando se utiliza “Explicit Congestion Notification”, (RFC 3168). El bit “ECE = ECN Echo”, le avisa al emisor que reduzca su velocidad de transmisión, y el emisor contesta con “CWR = Congestion Window Reduced”, señalando que ha recibido el mensaje ECE y ha reducido su velocidad.
- **URG = 1:** Se utiliza junto a “Urgent Pointer” para indicar que hay datos urgentes para ser leídos en el receptor y por lo tanto, debe saltarse el orden de secuencia normal en el receptor.
- **ACK = 1:** Significa que el segmento contiene un “acknowledgement” de un segmento recibido.
- **PSH = 1:** Indica que el receptor debe volcar los datos recibidos a la aplicación sin guardarlos en un buffer, (“PuSH”).
- **RST = 1:** “Reset”. Indica que debe recomenzar nuevamente la conexión, debido a algún problema.
- **SYN = 1:** Se utiliza para sincronizar el establecimiento de la conexión, “Connection Request”.
- **FIN = 1:** Indica que el transmisor no tiene más datos para transmitir y pueden desconectarse, “Disconnection Request”.



3 – ENCABEZAMIENTO TCP (continuación).

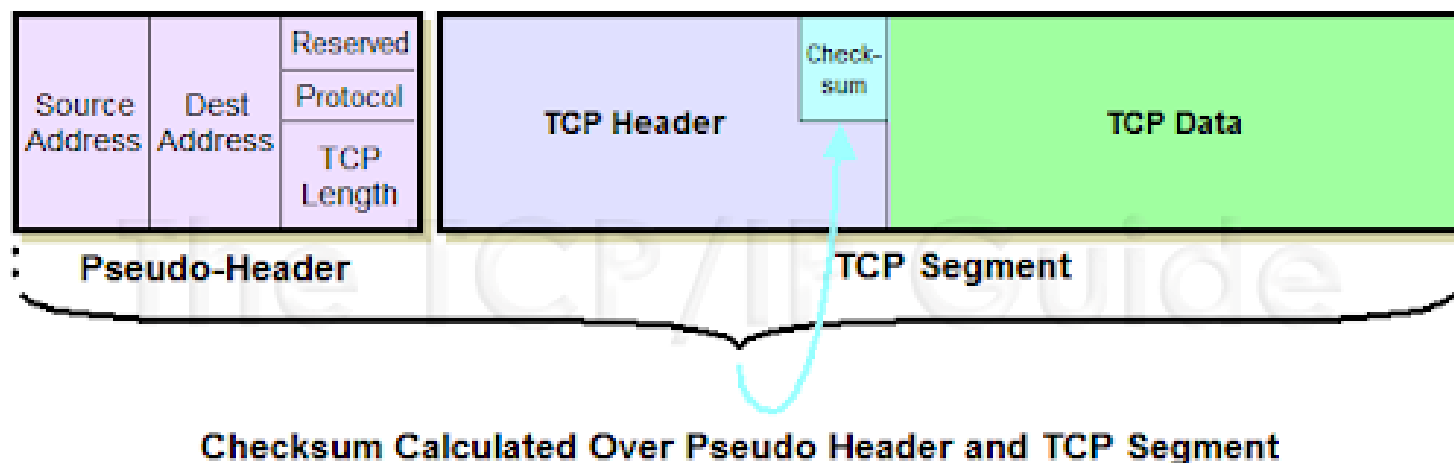
- **Windows Size:** Indica el tamaño de la ventana deslizante para control de flujo. Es decir, la cantidad de bytes que pueden enviarse sin recibir su “acknowledgement” correspondiente. El máximo es $2^{16} = 65.536$ bytes = 64 kB.
- **Checksum:** Es obligatorio en TCP. Controla el segmento completo TCP más el pseudoencabezamiento IP, igual que en UDP.
- **“Options”:** Estos campos opcionales añaden facilidades no contempladas en el encabezamiento normal, y pueden extenderlo hasta un máximo de 20 bytes más. Algunas opciones se definen al establecerse la conexión, otras en cambio, pueden definirse en cualquier momento, en medio de los segmentos de información.



The TCP header.

3 – ENCABEZAMIENTO TCP (continuación).

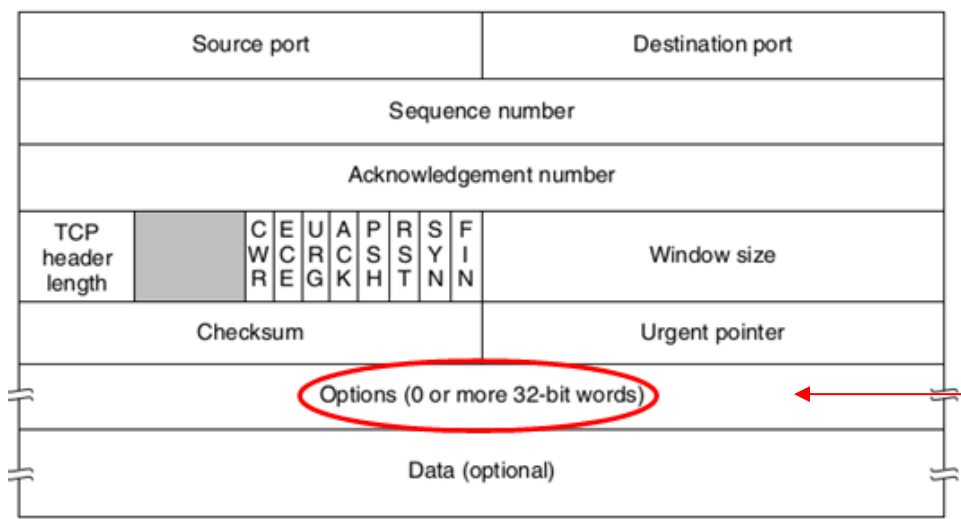
- El “Checksum” detecta errores en el segmento TCP completo y en algunos campos del paquete IP de capa 3, violando la estructura del modelo de capas.
- En la figura vemos los campos del pseudoencabezamiento IP que controla el checksum de TCP.



4 – CAMPOS OPCIONALES DE ENCABEZAMIENTO.

- ✓ **“Tamaño máximo de segmento”**: Es una de las opciones más usadas. Indica el máximo tamaño de segmento permitido. Por defecto, el tamaño es de **556 bytes** totales, que debe ser aceptado por cualquier host. El tamaño puede ser diferente en un sentido y en el sentido opuesto de la comunicación. Es determinante en el tamaño del buffer del receptor. (“Maximum Segment Size”).
- ✓ **“Tamaño de ventana deslizante”**: Es otra opción muy utilizada. Agranda el máximo de Ventana Deslizante, ya que en conexiones de gran velocidad y/o grandes retardos, el máximo por defecto de 16 bits (64 KB) para la Ventana es muy poco. (Por ejemplo, en una línea de 600 Mbps, una Ventana Deslizante de 64 KB se agota en 1 milisegundo y queda esperando “acknowledgements” ineficientemente. Si la conexión es transoceánica, donde el tiempo medio de propagación es de unos 50 milisegundos, resulta que el 98% del tiempo queda la línea sin usar). La opción “Window Scale” permite tamaños de ventanas de hasta 2^{30} bytes = 1 GB.
- ✓ **“Instante de salida”**: Es una opción que agrega el instante de tiempo (“Timestamp”) en que sale el segmento. Es utilizado para estimar el tiempo de viaje del segmento y así poder decidir si se da por perdido y retransmitirlo..
- ✓ **SACK = “confirmación selectiva”**: Otra opción muy utilizada. Permite al receptor enviar de una sola vez el rango de secuencia de los segmentos que han sido bien recibidos. (“SACK = Selective Acknowledgement”).

The TCP header.



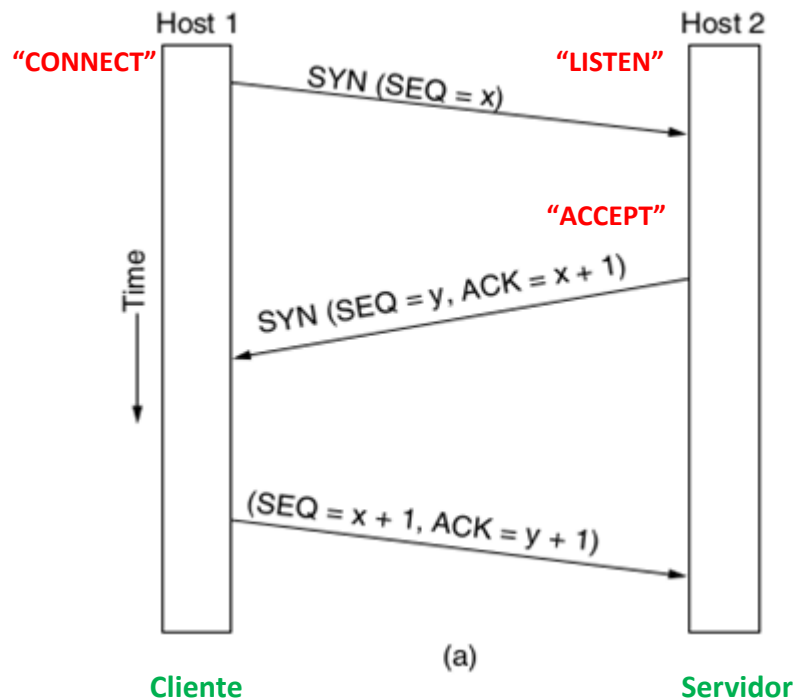
20 bytes como máximo,
(5 opciones).

5 – ESTABLECIMIENTO DE LA CONEXIÓN.

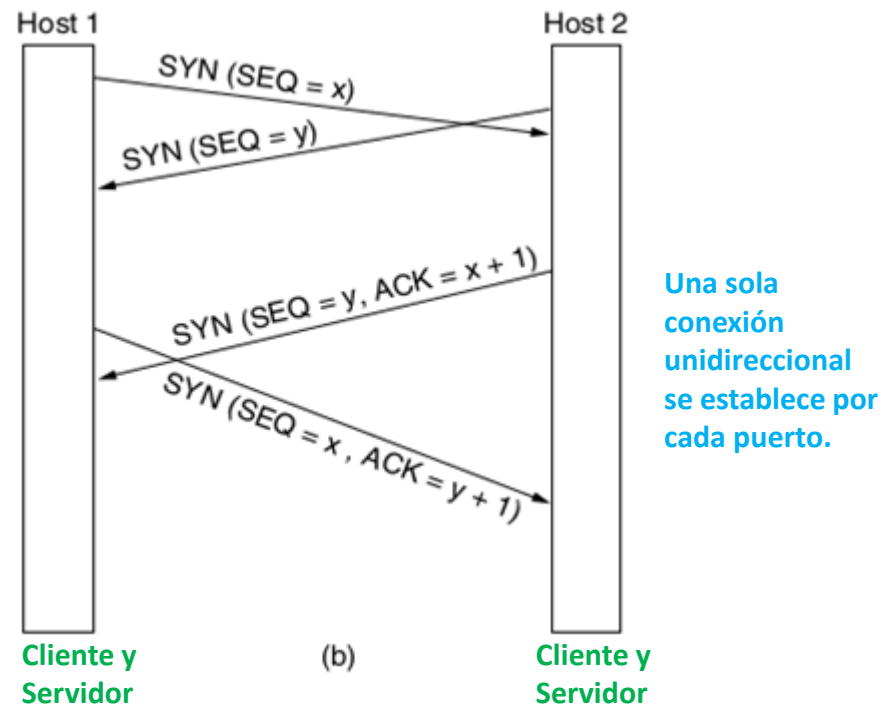
“Handshake de 3 vías”

- El servidor ejecuta la primitiva LISTEN.
 - El cliente ejecuta la primitiva CONNECT, enviando la dirección IP y el puerto en destino al que se quiere conectar, el tamaño de segmento máximo que admite, y algún otro dato adicional. Lleva el bit SYN en “1”, el bit ACK en “0”, esperando una respuesta del otro lado.
- Por razones de seguridad hoy se emplea criptografía también en estos procesos para evitar conexiones maliciosas. Ver RFC 4987.

Conexión normal

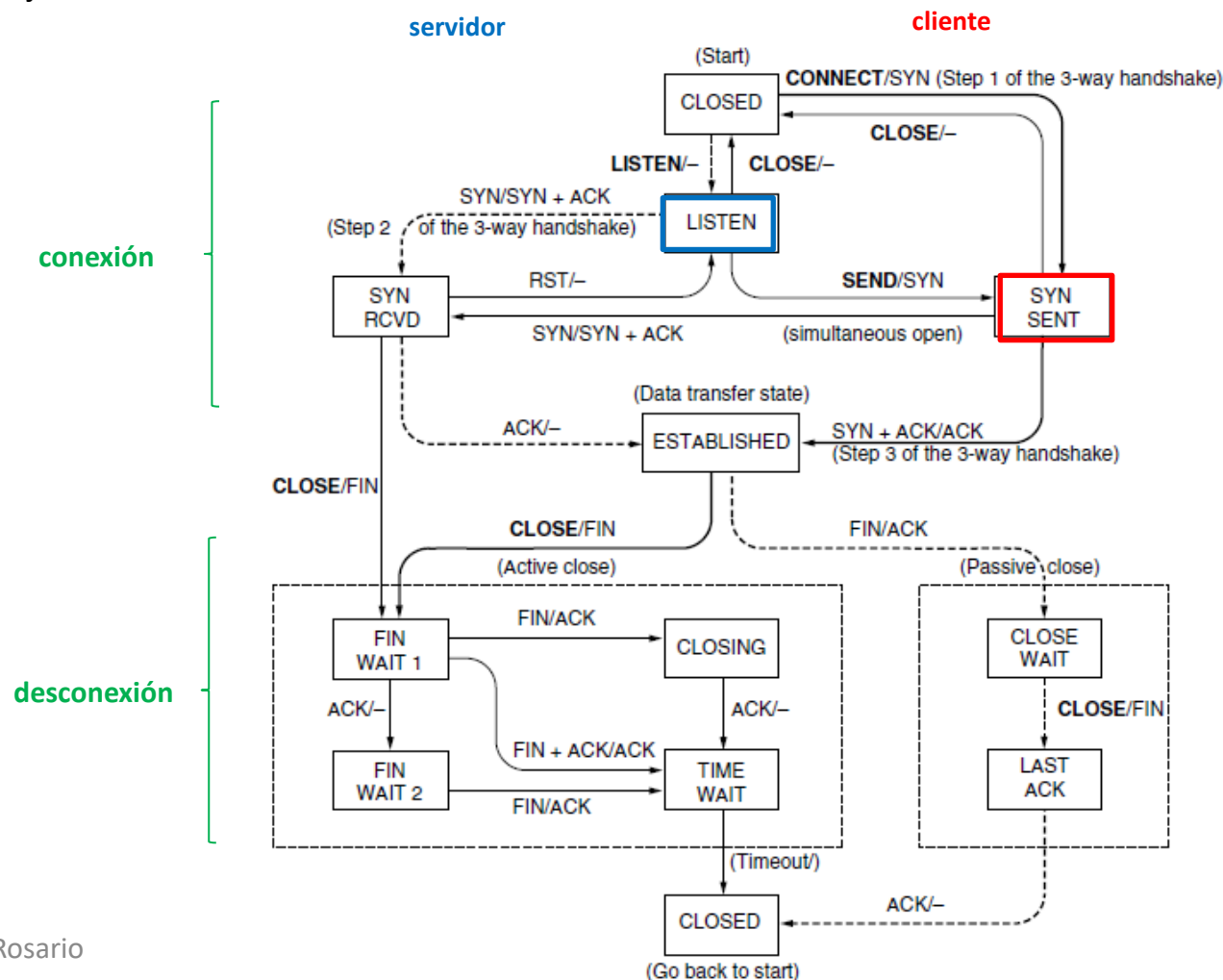


Intento de conexión simultánea por ambos lados



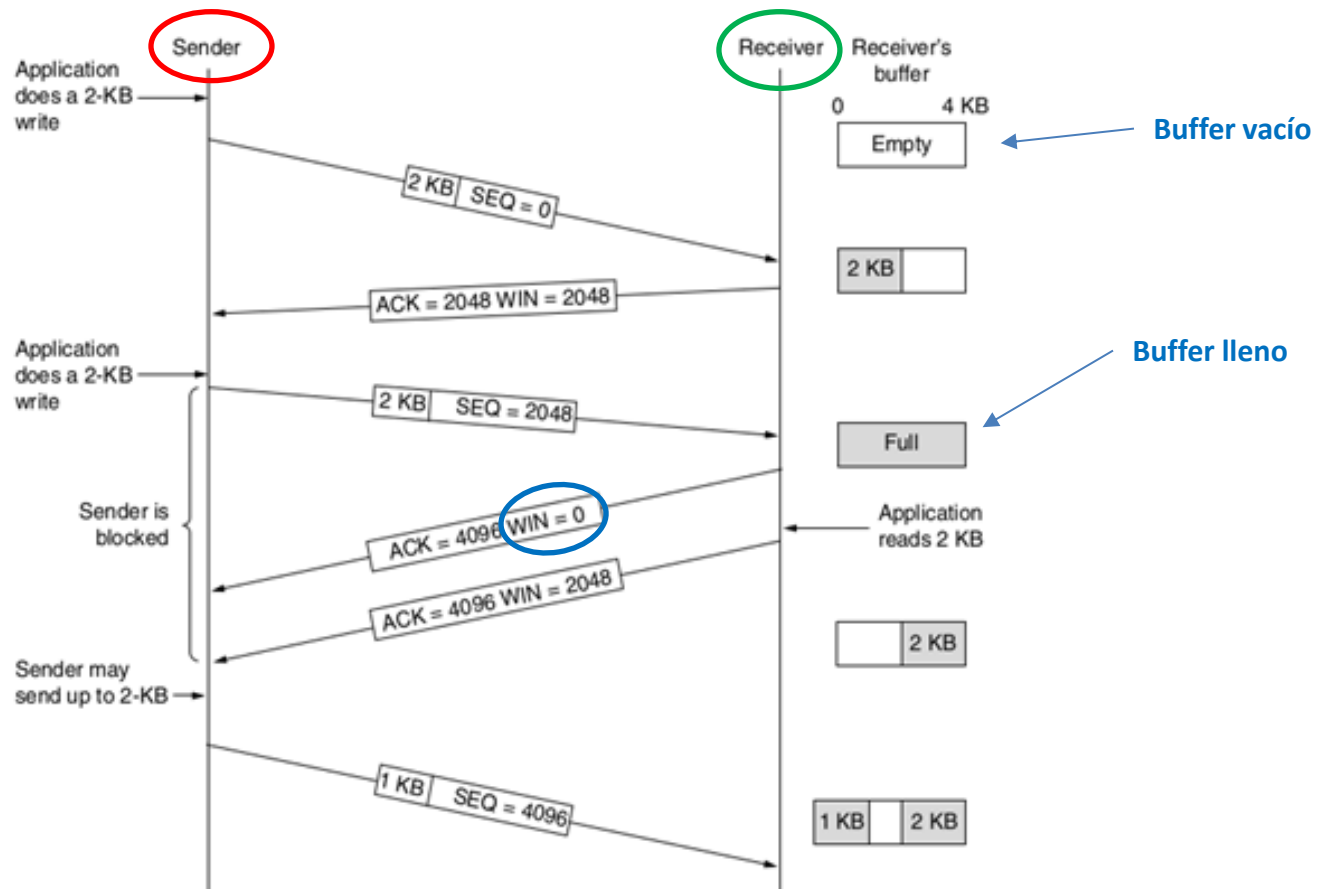
6 – SECUENCIA DE ESTADOS DE LA COMUNICACIÓN.

- **Diagrama de estados:** En la figura se observan los estados por los que pasa una conexión TCP. El “handshake de 3 vías” para la conexión y para la desconexión. Las líneas gruesas indican la secuencia normal de un cliente. Las líneas finas marcan secuencias inusuales. Para la desconexión se necesita desconectar ambos sentidos de la comunicación y, por lo tanto, se envían 4 primitivas. Es decir, un “FIN” y su “ACK” en cada dirección. Si se pierde uno de estos segmentos la desconexión se efectúa igual con un temporizador que expira su cuenta, evitando así el problema de “los dos ejércitos”.



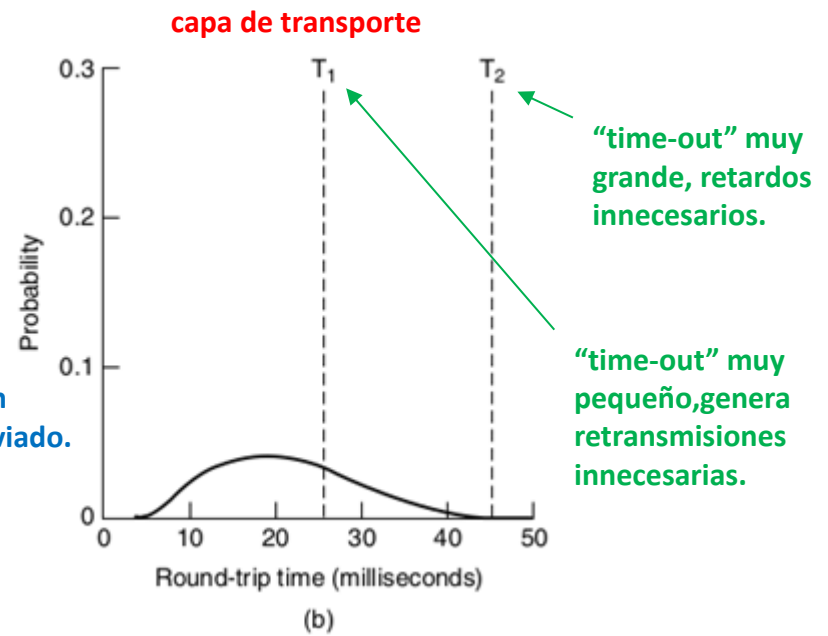
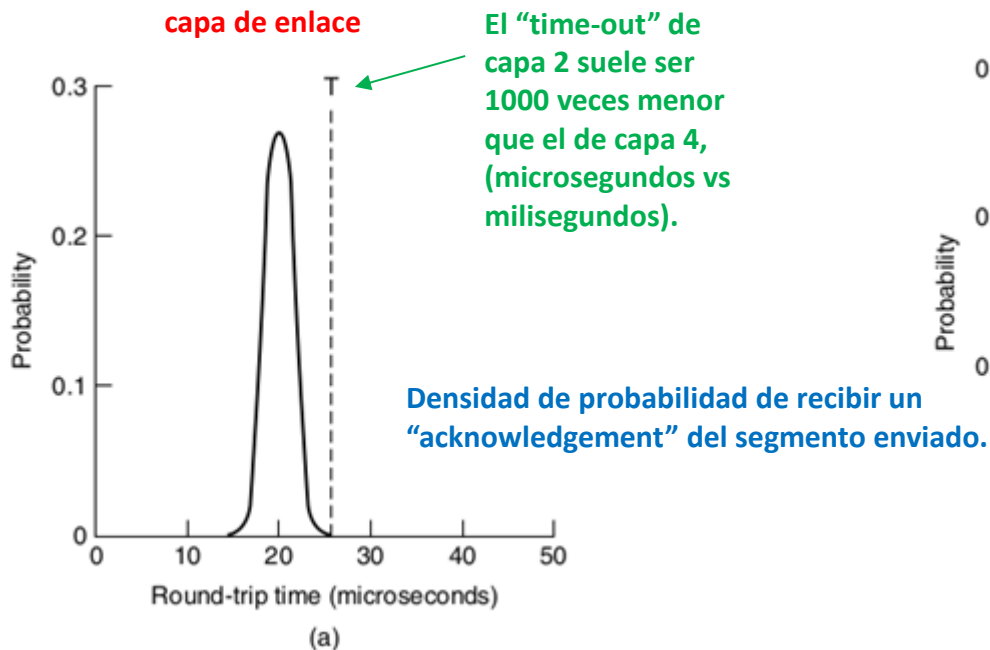
7 – VENTANA DESLIZANTE.

- En TCP se utiliza una ventana deslizante para el control de flujo, como hemos mencionado.
- Ejemplo: En la figura se observa un receptor con buffer de 4 KB comunicándose con un emisor que le envía segmentos de 2 KB. El receptor contesta enviando un “acknowledgement” del segmento recibido y el espacio libre en su buffer. Luego de recibir dos segmentos el buffer se llena. Para mayor eficiencia, se intenta enviar segmentos grandes cuando las aplicaciones permitan cierto retardo, (algoritmo de Nagle).



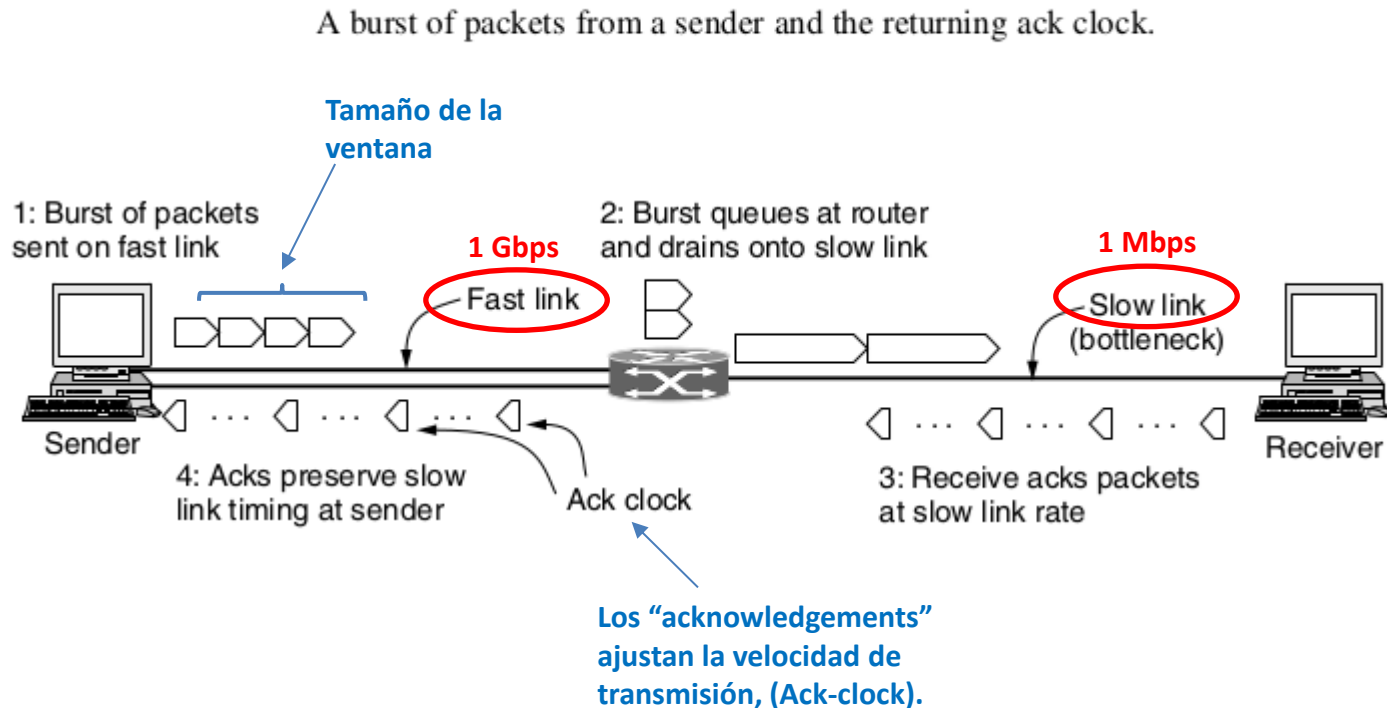
8 – TEMPORIZADORES.

- Varios temporizadores se utilizan en TCP. El más importante es el que decide una retransmisión del segmento, al considerarlo perdido. Es algo más complicado que en capa 2, como se aprecia en las figuras porque hay menos desviación estándar en la capa de Enlace. A la derecha se ve la situación en capa 4, donde la probabilidad de recibir un “acknowledgement” está más distribuida en el tiempo.
- Temporizador dinámico:** La mejor solución es ajustar el “time-out” en forma dinámica, adaptándolo al estado de congestión de la red. El tiempo mínimo suele ser 1 segundo en capa 4. La RFC 2988 analiza estos temporizadores.



9 – CONTROL DE CONGESTIÓN.

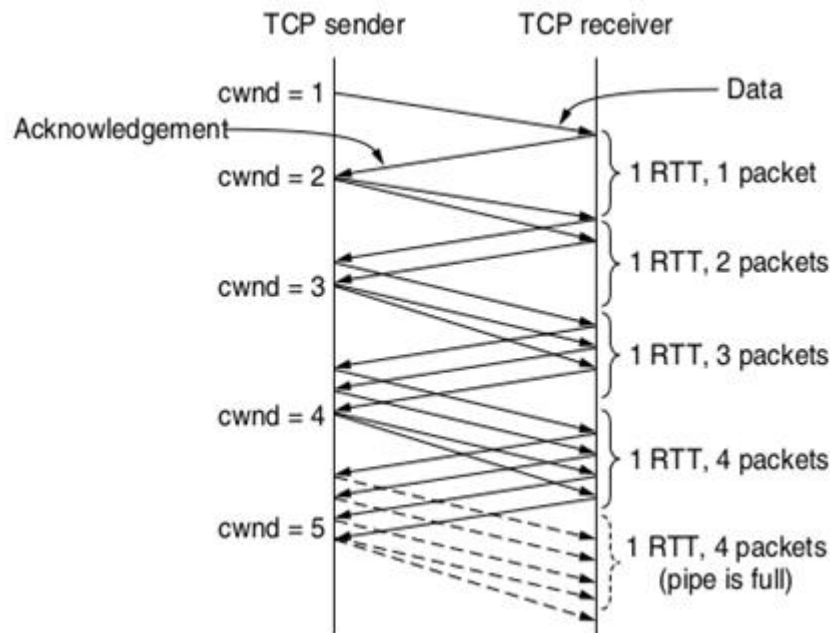
- TCP utiliza una “ventana deslizante” definida como una cierta cantidad de bytes que está viajando por la red sin su correspondiente “acknowledgement”. TCP ajusta dinámicamente el tamaño de esta ventana según la regla AIMD (Additive Increment, Multiplicative Decrement) que hemos visto.
- La confirmación ajusta el reloj:** Los “acknowledgements” recibidos determinan la velocidad adecuada para enviar nuevos segmentos. Por ejemplo, en la figura se observa un primer enlace muy rápido seguido de un enlace más lento después del router. Por lo tanto, el emisor envía sólo una primera ráfaga pequeña de segmentos y se detiene a esperar que retornen las confirmaciones del receptor. Este proceso evita una congestión y se denomina “Acknowledgement-Clock”.



9 – CONTROL DE CONGESTIÓN (continuación).

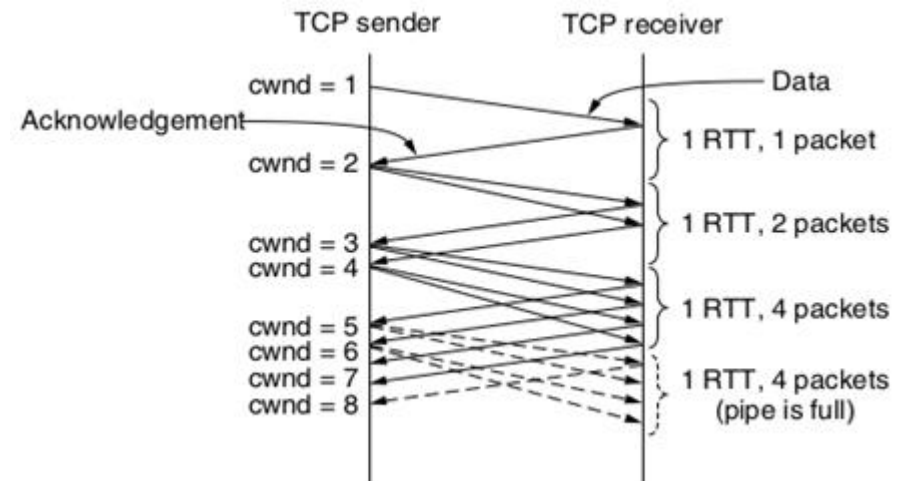
- **Algoritmo de incremento aditivo:** Una forma de ajustar el tamaño de la ventana deslizante es aumentar en una unidad su tamaño con cada retardo ida y vuelta, según se observa en la figura izquierda.
- **Algoritmo “slow start”:** Este algoritmo es mejor aún. Consiste en aumentar al doble el tamaño de la ventana con cada confirmación del receptor, es decir, midiendo el RTT (retardo ida y vuelta). Comienza lento, enviando un primer segmento, y si su acuse de recibo retorna antes del tiempo preestablecido, la ventana de congestión se incrementa al doble, enviando 2 segmentos. Y así sucesivamente. Este comportamiento se observa en la figura de la derecha. Más temprano o más tarde habrá congestión y entonces se disparará el “time-out” de retransmisión de algún segmento. Cuando esto suceda, la ventana de congestión se achicará a la mitad. (“slow-start algorithm”).

Incremento aditivo



Additive increase from an initial congestion window of one segment.

Algoritmo “slow-start”



Slow start from an initial congestion window of one segment.

RTT = “Round trip time” = retardo ida y vuelta.

9 – CONTROL DE CONGESTIÓN (continuación).

- **TCP Tahoe:** En 1988 se implementó este algoritmo graficado en la figura izquierda. La conexión TCP comienza en “slow-start” hasta llegar a un cierto tamaño de ventana (Threshold) y luego continúa evolucionando con incrementos aditivos, más suaves. Al recibirse por triplicado un aviso de que falta un segmento, se considera que se ha perdido e indica que hay una congestión. El algoritmo comienza nuevamente pero ahora con un tamaño de ventana deslizante igual a la mitad del anterior.
- **TCP Reno:** Es una versión posterior de TCP que se implementó en 1990 y es parecido al anterior TCP Tahoe, sólo que se agrega un “fast recovery” luego de perder un paquete, retomando desde un tamaño de ventana igual a la mitad del tamaño anterior. Es decir, se utiliza la regla AIMD, (Additive Increase, Multiplicative Decrease) después de la primera pérdida. Ver figura derecha. Las más recientes versiones, “Cubic TCP” en Linux y “Compound TCP” en Windows. son muy parecidas a TCP Reno.

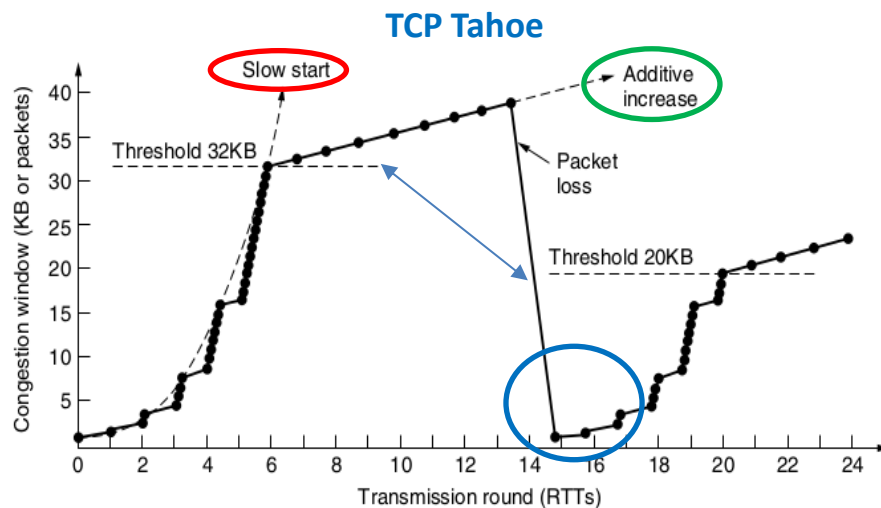
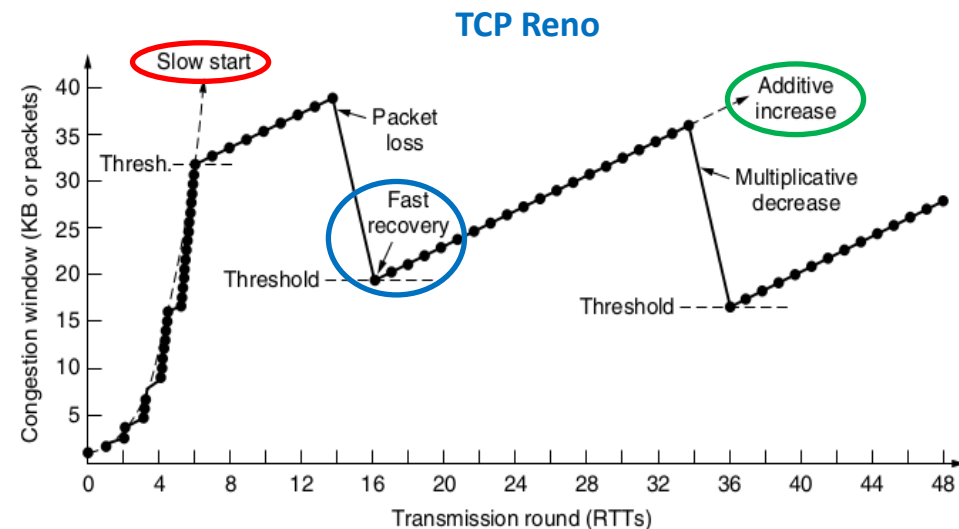


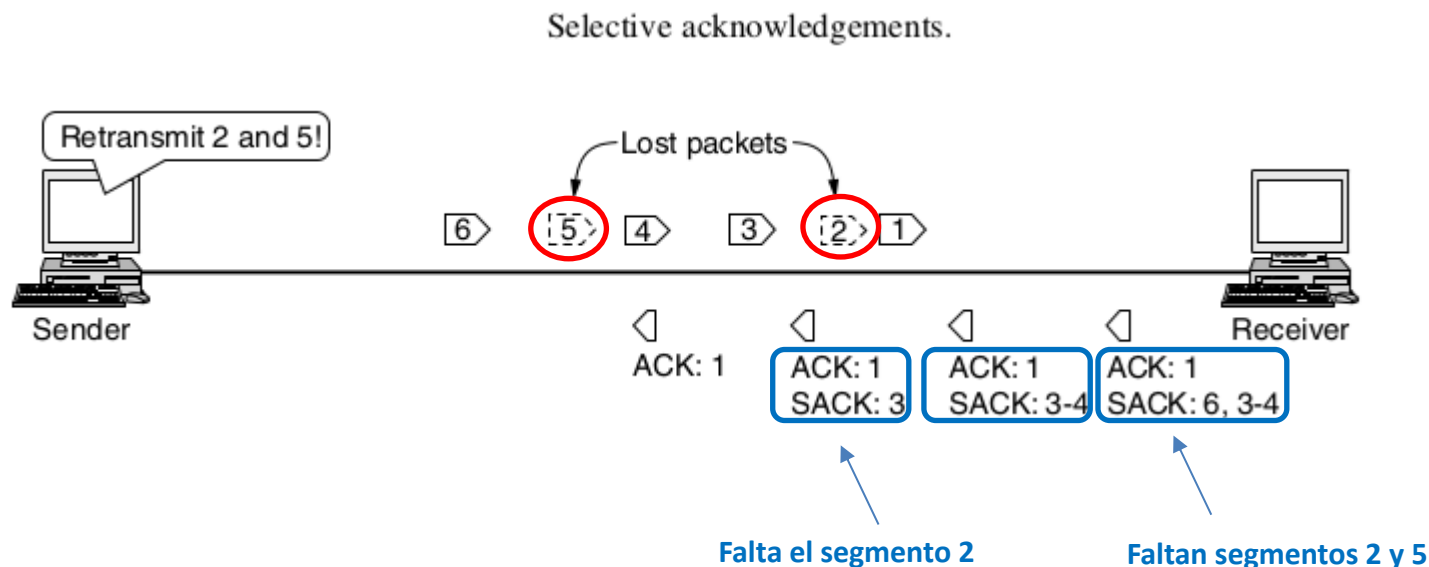
Figure 6-46. Slow start followed by additive increase in TCP Tahoe.



Fast recovery and the sawtooth pattern of TCP Reno.

10 – “ACKNOWLEDGEMENT” SELECTIVO (SACK).

- **Confirmación selectiva:** Gran parte del funcionamiento de TCP pasa por determinar cuando se ha perdido un segmento analizando los “acknowledgements” que van llegando. Empleando la opción SACK, (“Acknowledgements” Selectivos) es sencillo saber qué segmento retransmitir. SACK está descrito en la RFC 2883 y la RFC 3517 especifica el control de flujo con SACK.
- **Ejemplo:** En la figura se observa como funciona SACK. El receptor envía un SACK cuando no ha recibido el segmento anterior. Y puede enviar hasta 3 rangos de segmentos recibidos con faltantes. En el ejemplo, se han perdido los segmentos 2 y 5, por lo tanto, corresponde enviar un “acknowledgement” por el segmento 1 y un SACK de los segmentos 3,4 y 6.



11- BIBLIOGRAFÍA.

- Tanenbaum, Andrew y Wetherall, David. “Computer Networks”. 5th edition, 2011. Pearson Prentice Hall Edition.